Real-Time Color Quantization Using Octree Structures for Memory-Efficient Image and Video Processing

Tuomas Haapasalo, Pedro Marinho, Miguel Mateus

June 8, 2026

Abstract

High-resolution image and video processing requires massive amounts of memory. While modern screens can easily display millions of colors using 24-bit to 32-bit color depth, many systems cannot always afford the RAM, Video RAM (VRAM), or bandwidth required to store, process, or transmit that data in real time.

This project addresses the challenge by implementing and comparing different **Color Quantization algorithms**. The main focus of the project is to implement and evaluate an **Octree-based quantization approach** and compare the Octree approach against other common quantization algorithms, e.g., **Uniform Quantization**, **Median-Cut**, **K-Means**, and **Self-Organizing Maps (SOM)**. In addition, hybrid approaches such as **Octree-SOM** and **Octree-K-Means** are evaluated to study whether Octree initialization can improve the quality or efficiency of more advanced methods.

In addition to evaluating static image quantization performance, the project also investigates whether the proposed algorithms can be applied in practical real-time systems. To demonstrate this, two prototype applications were developed. The first application performs real-time quantization of a live camera feed, while the second integrates quantization into a live game and screen capture pipeline. These applications provide a practical evaluation of whether the algorithms can maintain sufficient frame rates for interactive use and complement the traditional image quality and execution time benchmarks.

The goal is to determine which algorithm is best suited for different practical needs: visual accuracy, computational speed, and the tradeoff between the two. Furthermore, the project investigates whether Octree-based quantization can be used in real-time applications through live camera processing and game/screen capture integration. This is important in environments with hardware or bandwidth limitations, such as:

- **Embedded Systems:** Smartwatches and IoT devices where RAM must be reserved for background processes rather than UI assets.
- **Game Development:** Reducing texture color depth frees up VRAM, allowing the engine to load more objects into a scene simultaneously.
- **Bandwidth Optimization:** Reducing the number of colors in a live video stream can make real-time transmission more feasible under poor network conditions.

Contents

1 Introduction

1.1 Motivation

Digital images and videos are commonly represented using high color depths, such as 24-bit RGB, where each pixel can represent more than 16 million possible colors. This gives visually rich results, but it also creates a significant memory and bandwidth cost. For a single high-resolution image this cost may be acceptable, but in video processing the same cost is repeated for every frame. As a result, real-time image and video applications can quickly become limited by RAM, Video RAM (VRAM), cache usage, or network bandwidth.

This problem is especially important in systems where hardware resources are limited or where many visual assets must be processed at the same time. Embedded devices, such as smartwatches and IoT displays, often have strict memory limits and cannot reserve large amounts of RAM for image data alone. Similarly, in game development, high-resolution textures consume VRAM, which limits how many objects, environments, and visual effects can be loaded simultaneously. In networked systems, such as live video streaming, transmitting full-color video can require more bandwidth than is available under poor network conditions.

Color quantization addresses this challenge by reducing the number of colors used to represent an image while preserving as much visual quality as possible. Instead of storing every pixel using the full original color range, the image is represented using a smaller color palette. This means that a 24-bit image can, for example, be approximated using 8-bit or even 4-bit color indices. Although this introduces some loss of visual detail, the trade-off can be worthwhile when the goal is real-time performance, lower memory usage, or more efficient transmission.

1.2 Problem

The central problem in this project is how to reduce the number of colors in an image or video frame while keeping the result visually acceptable and computationally efficient. A naive approach would be to simply round color values or select colors uniformly from the RGB space. However, this often produces poor visual results because the selected colors may not match the actual distribution of colors in the image. For example, an image containing mostly blue sky and green grass should allocate more palette entries to blues and greens than to colors that do not appear in the image.

A good color quantization algorithm must therefore solve two related tasks. First, it must analyze the input image and construct a representative palette of limited size. Second, it must map each original pixel to the closest or most suitable color in that palette. These tasks must be performed under practical constraints: the algorithm should not use excessive memory, should be fast enough for real-time or near-real-time processing, and should scale reasonably to high-resolution images and video frames.

The difficulty becomes greater in video processing, where the algorithm may need to process many frames per second. If palette construction is too slow or memory-intensive, the system cannot operate in real time. If the palette is too small or poorly chosen, the output image may contain visible artifacts such as banding, unnatural color transitions, or loss of important details. Therefore, the goal is not only to minimize memory usage, but to find a balance between compression, speed, and visual quality.

1.3 Related Work

Color quantization has been studied through several different algorithmic approaches. These methods differ mainly in how they choose a reduced color palette and how much computation they require. An overview of each method is presented in Table 1.3.

Table 1: Overview of the most common color quantization algorithms.

	Description	Advantages	Disadvantages
Uniform quantization	Split into N equally sized ranges.	Simple and fast.	Poor visual quality because it does not adapt to the image colors.
Median-Cut	Recursively splits the color space into boxes with similar number of pixels.	Adapts to the image content and gives a good speed-quality tradeoff.	Requires sorting and repeated partitioning, which can be expensive.
K-Means	Groups image colors into clusters and uses the cluster centers as the palette.	Usually produces very accurate palettes because it directly optimizes color groups.	Computationally expensive for large or high-quality images.
SOM	Uses a self-organizing map that updates neuron weights based on image pixels.	Can capture complex color distributions, especially with larger palettes.	Sensitive to initialization. Performs poorly with very small palettes.
Octree-Baseline	Builds palette using an octree representation.	Fast, deterministic, and well-suited for incremental or real-time processing.	Does not directly account for color frequency.
Octree-SOM	Initializes SOM weights using an octree generated palette.	Gives SOM a better starting palette and can improve stability.	Adds complexity and slowness compared to standard SOM.
Octree-K-Means	Octree palette is refined with K means.	Combines fast Octree initialization with high-quality K-Means refinement.	Adds complexity and slowness compared to standard Octree.
Octree-Euclidean	Image colors are mapped from the octree structure using euclidean distance.	Improves visual quality over simple Octree remapping.	Slower than baseline Octree because each pixel needs nearest-color comparison.

One simple approach is uniform quantization. In this method, the RGB color space is divided into equally sized regions, and each color is mapped to the nearest representative value. Uniform quantization is very fast and easy to implement because it does not need to analyze the actual color distribution of the image. However, this is also its main weakness. Since the palette is chosen without considering which colors are actually common in the image, many palette entries may be wasted on unused colors, while visually important color regions may receive too few representatives. As a result, uniform quantization often produces poor visual quality, especially in images with smooth gradients or dominant color regions.

Another classical method is the median-cut algorithm introduced by Heckbert [1]. Median cut analyzes the distribution of colors in the image and recursively splits the color space into boxes that contain approximately equal numbers of pixels. This usually produces better visual results than uniform quantization because the palette adapts to the image content. However, the algorithm requires sorting and repeatedly partitioning color data, which can become expensive when processing high-resolution images or video frames in real time.

K-Means clustering is another possible approach to color quantization [2]. It treats the image colors as data points and attempts to find cluster centers that represent the final color palette. This can produce high-quality palettes because the selected colors are optimized based on the actual input image. However, K-Means is iterative and computationally expensive, especially when the number of pixels and palette colors is large. In addition, the algorithm may require several iterations before convergence and is not guaranteed to find the global optimum. For this reason, K-Means is usually not suitable for real-time video quantization without additional approximations or acceleration.

Self-Organizing Maps, or SOMs, provide another adaptive approach to color quantization [3]. A SOM is a type of neural network that learns a set of representative color vectors from the image data. During training, the map gradually adjusts its nodes so that similar colors are represented by nearby nodes. This can produce good palettes, especially when the desired number of output colors is large. However, SOM-based quantization is sensitive to initialization and learning parameters, and it can perform poorly when the number of output colors is very small. Since SOM training is also iterative, it may be less suitable for strict real-time applications unless the implementation is carefully optimized.

Octree color quantization provides a practical compromise between these approaches [4]. It represents the RGB color space using a hierarchical tree structure, where each level of the tree corresponds to a more detailed subdivision of color values. Similar colors are grouped into shared branches, and the tree can be reduced until the required number of palette colors is reached. Compared with uniform quantization, the octree method adapts better to the actual colors in the image. Compared with K-Means and SOM, it avoids costly iterative optimization. Compared with median cut, it offers a natural data structure for incremental processing and palette reduction, making it especially interesting for memory-efficient real-time image and video processing.

In addition to the baseline octree method, this project also considers hybrid octree-based approaches. In the Octree-SOM method, the palette generated by the octree is used to initialize the SOM [5]. The motivation is that the octree can provide a more image-specific starting point than random initialization, while the SOM can further adjust the palette based on the image colors. This may improve palette quality, but it also adds extra computational cost and complexity compared with the standard SOM method.

Similarly, in the Octree-K-Means method, the octree-generated palette is used as the initial set of cluster centers for K-Means [6]. Since standard K-Means can be sensitive to initialization, using an octree palette may help the algorithm start from more meaningful color centers. The K-Means step can then refine the palette to improve visual quality. However, this hybrid method is more expensive than baseline octree quantization because it still requires an iterative clustering stage.

Finally, there exist the Octree-Euclidean variant that differs from the baseline Octree method by the mapping step. Both methods construct the reduced colour palette using octree quantization, but they differ in the final pixel mapping step. In the baseline method, pixels are mapped according to their corresponding octree leaf representative. In Octree-Euclidean, each original RGB pixel is instead assigned to the palette colour with the smallest Euclidean distance,

$$d(x, p) = (R_x - R_p)^2 + (G_x - G_p)^2 + (B_x - B_p)^2.$$

Thus, Octree-Euclidean keeps the same palette construction principle, but uses nearest-colour reconstruction during the final mapping stage.

2 Octree Color Quantization

Octree color quantization is a hierarchical method for reducing the number of colors in an image while preserving its main visual appearance. It is especially suitable for 24-bit RGB images, because each pixel consists of three 8-bit channels: red, green, and blue. The octree structure directly follows this representation by using one bit from each channel at every tree level [4].

2.1 Overview

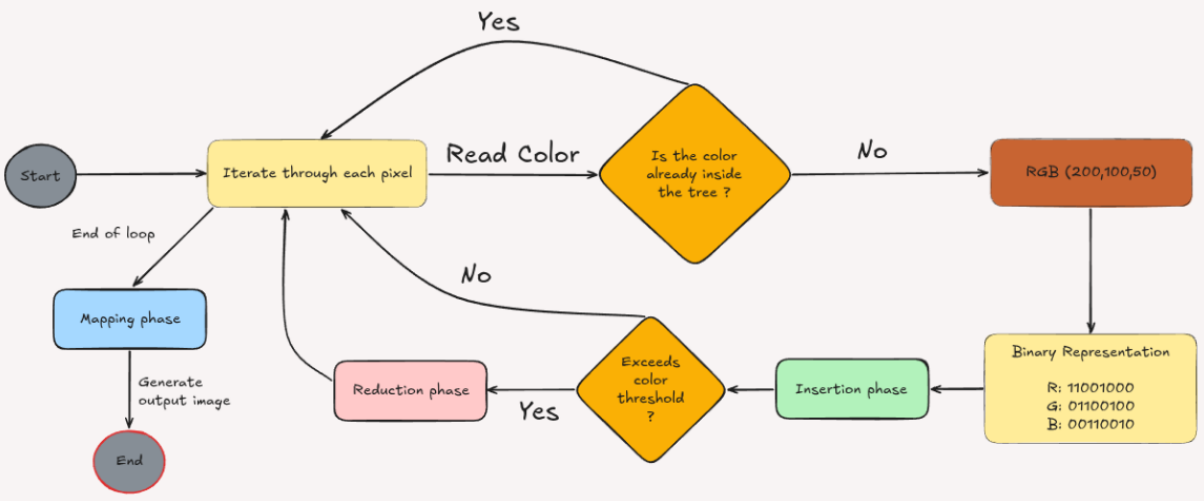

Figure 1: Overview of the octree color quantization algorithm.

The octree algorithm can be divided into three main phases: insertion, reduction, and mapping. First, the algorithm loops through every pixel of the original image and inserts its RGB color into the octree. If the color already exists in the tree, the existing node is updated by increasing its pixel count and adding the pixel's red, green, and blue values to the node's accumulated RGB sums. These statistics allow the algorithm to later compute representative palette colors as weighted averages, where more frequent colors have a larger effect on the final result. If the color does not yet exist in the tree, a new path is created based on the binary representation of the color.

After insertion, the tree may contain more colors than the desired target palette size. In that case, the reduction phase merges nodes until the number of remaining leaf nodes matches the target number of colors. Finally, in the mapping phase, every original pixel is replaced with one of the colors from the generated octree palette.

2.2 Insertion Phase

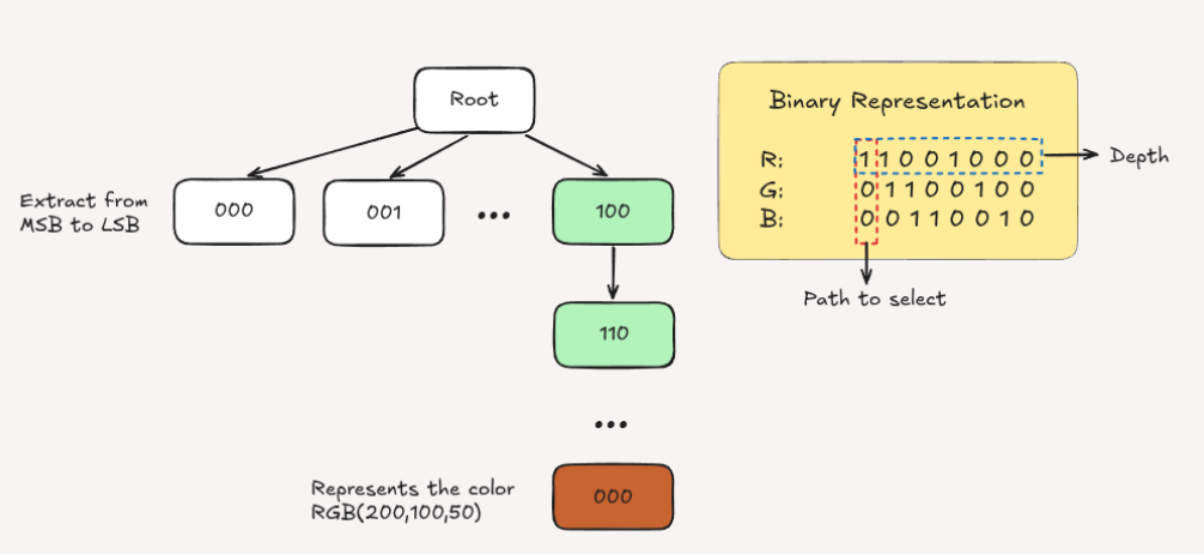

Figure 2: Insertion of an RGB color into the octree using its binary representation.

In the insertion phase, each pixel is read from the input image and converted into its binary RGB representation. Since each channel has 8 bits, a 24-bit RGB color can be represented as

$$R = r_7r_6r_5r_4r_3r_2r_1r_0, \quad G = g_7g_6g_5g_4g_3g_2g_1g_0, \quad B = b_7b_6b_5b_4b_3b_2b_1b_0.$$

At each level of the octree, one bit from each channel is used to select one of eight possible child nodes. For example, at the first level, the algorithm uses the most significant bits (r_7, g_7, b_7). At the second level, it uses (r_6, g_6, b_6), and so on. Therefore, the child index at depth i is determined by

$$(r_i, g_i, b_i).$$

This means that the upper levels of the tree represent large color regions, while the deeper levels represent smaller color differences since the most significant bit has the biggest effect on the numeric color value. For instance, the color $\text{RGB}(200, 100, 50)$ can be inserted by reading the bits of each channel from the most significant bit to the least significant bit. The selected path through the tree uniquely determines the region of RGB space to which the color belongs.

Each leaf node stores information about the colors assigned to it, such as the sum of red, green, and blue values and the number of pixels. These values are later used to calculate the average representative color for that node.

2.3 Reduction Phase

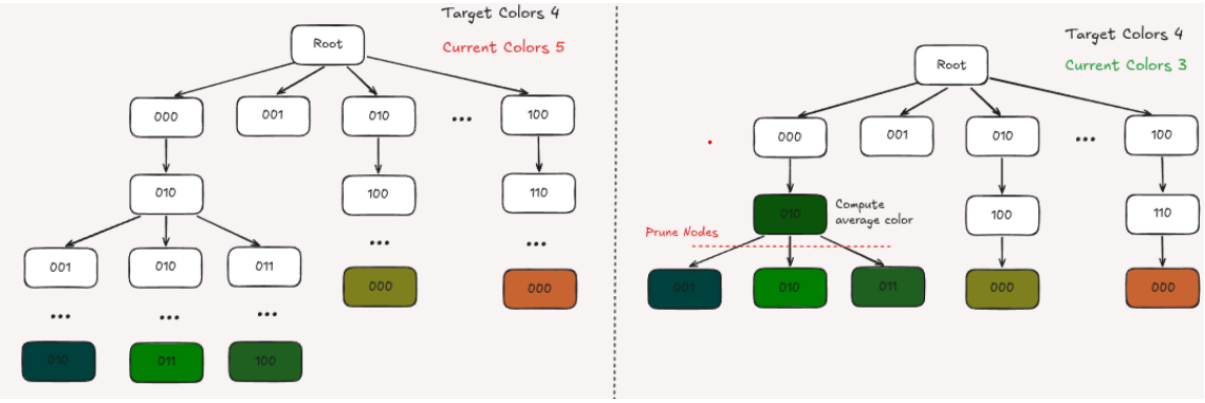

Figure 3: Reduction phase where leaf nodes are merged to reach the target number of colors.

The reduction phase is needed when the number of leaf nodes exceeds the desired number of output colors. Each leaf node represents a possible palette color, so reducing the number of leaves reduces the final palette size.

Reduction is typically performed from the deepest level of the tree upwards. This is useful because the deepest nodes correspond to the least significant bits of the RGB values. In other words, they describe small color variations rather than major color differences. By merging these deeper nodes first, the algorithm removes fine details while preserving the main color structure of the image.

When nodes are pruned, their color information is merged into their parent node. The parent node then becomes a new leaf node, and its representative color is computed as the average of all colors that were contained in the removed child nodes. This process continues until the number of leaf nodes is less than or equal to the target number of colors.

This hierarchical pruning is what makes the final mapping phase so highly efficient. Because an octree is bounded by a maximum depth of 8, color lookups are inherently fast O(1) operations. Reduction further optimizes this by truncating the traversal paths, allowing the mapping phase to find the nearest palette color using direct, bitwise traversal in just a few operations, without needing to search the entire color space.

2.4 Mapping Phase

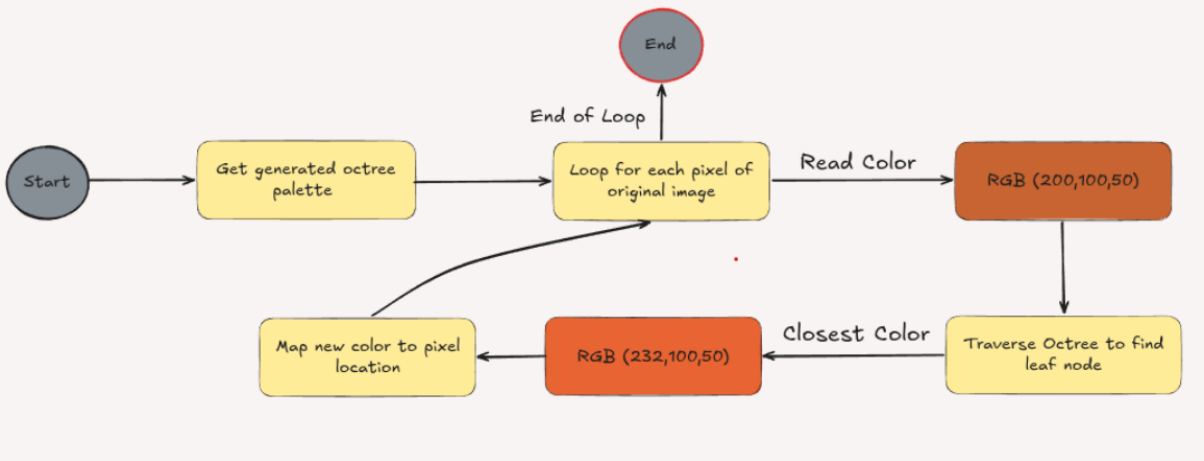

Figure 4: Mapping phase where each original pixel is replaced by its corresponding octree palette color.

After the reduction phase, the remaining leaf nodes form the final color palette. However, the palette alone is not yet the final image. The algorithm must still assign a palette color to every pixel in the original image.

In the mapping phase, the algorithm loops through each original pixel again. For each pixel, the octree is traversed using the pixel's RGB binary representation, in the same way as during insertion. The traversal leads to a remaining leaf node, and the original pixel color is replaced by the representative color stored in that leaf.

This produces the final quantized image. The visual quality depends on how well the remaining leaf nodes represent the original color distribution. Since the octree keeps the most significant color information in the upper levels and removes smaller variations from the lower levels, it provides an efficient and natural way to reduce 24-bit RGB images to a smaller color palette.

3 Implementation

3.1 Strategy

To demonstrate the feasibility and scalability of the quantization methods, the implementation follows a four-stage incremental path. Each phase increases the difficulty of the input source, starting from static images and ending with live interactive screen capture.

1. **Phase 1: Static Image Quantization.** The first phase focuses on individual images. A high-resolution image is loaded, converted into an RGB pixel array, and processed using the selected quantization algorithm, Figure 5.
2. **Phase 2: Pre-Rendered Video Quantization.** The second phase extends the static image pipeline to video. A video file is read frame by frame using OpenCV, each frame is converted from BGR to RGB, quantized, converted back to BGR, and written into a new output video. This phase tests how the algorithms behave when repeated over many consecutive frames.
3. **Phase 3: Real-Time Camera Feed.** The third phase connects the quantization algorithms to a live webcam feed, Figure 6. The system captures frames continuously, applies the selected quantization algorithm, and displays the processed result in a graphical interface. The user can change the algorithm, the number of output colors, and the camera resolution while the program is running. This phase stress-tests whether the implementation can maintain interactive frame rates during live processing.
4. **Phase 4: Game and Screen Capture Integration.** The final phase applies the quantization pipeline to an interactive window, such as a game or graphical application, Figure 7. The selected image is captured, quantized, and then sent as the processed output. For better performance, the algorithm can also downscale the image before quantization and then upscale the result afterwards, reducing computation while preserving the pixelated retro effect. This phase demonstrates the use of color quantization as a real-time post-processing effect.

Figure 5: Static Image Quantization.

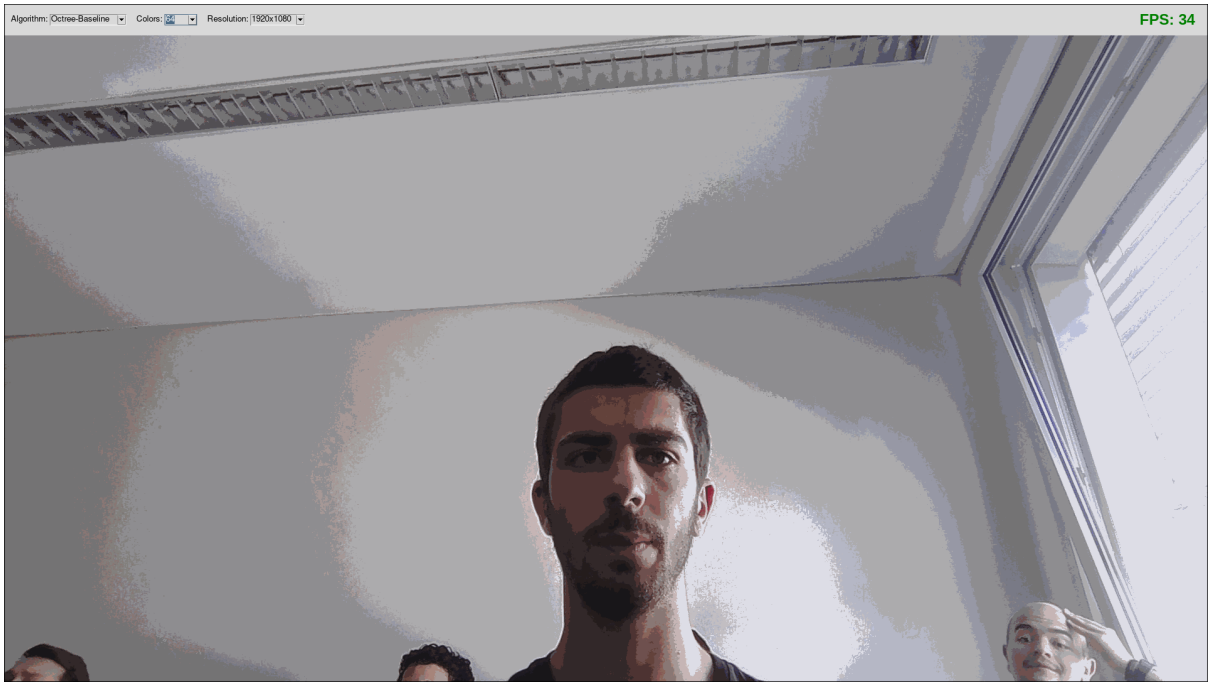

Figure 6: Real-Time Camera Feed

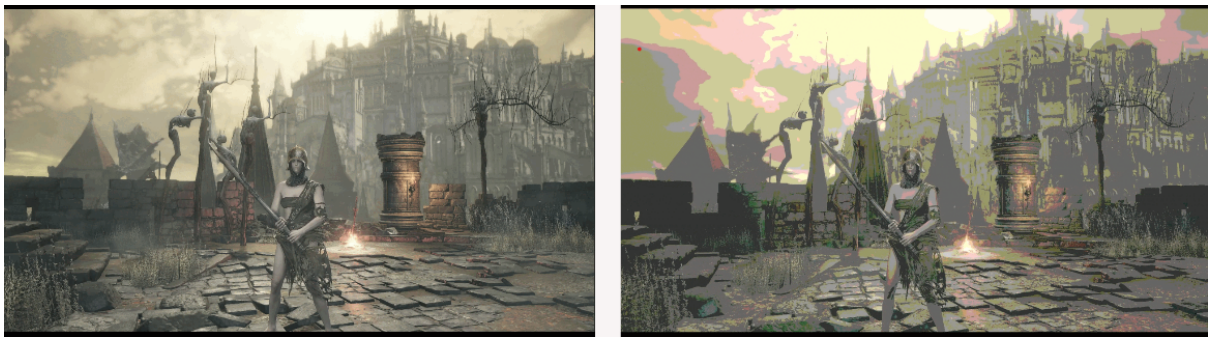

Figure 7: Game and screen capture integration.

3.2 Implementation

The implementation is divided into a Python control layer and a C++ performance layer. Python is used for loading images and videos, managing the command-line interface, running the different project phases, saving outputs, and generating benchmark statistics. The computationally expensive algorithms are implemented in C++ and exposed to Python through a shared library loaded with `ctypes`. This allows the project to keep a simple Python interface while using faster native code for pixel-level processing. All the source code is provided in the linked Github repository.

3.3 Functionalities

The implementation provides the following main functionalities which are present in the makefile:

- **Single image quantization:** The system can quantize one input image using a selected target number of colors. The result is saved as an output image.
- **Batch image quantization:** The system can process an entire directory of images. This

is used for benchmarking the algorithms over several test images instead of relying on a single example.

- **Multiple algorithm comparison:** The implementation supports Octree-Baseline, Median-Cut, K-Means, SOM, Octree-SOM, and Octree-Live and Octree-K-Means.
- **Multiple palette sizes:** The benchmarking pipeline evaluates the algorithms using different target palette sizes, starting from 8 colors and doubling up to the selected maximum, such as 256 colors.
- **Benchmark logging:** For each processed image and algorithm, the system records execution time, final color count, MAE, MSE and SSIM into CSV files. We use SSIM as the defining metric for accuracy.
- **Statistics and chart generation:** The system can generate comparison charts for each image, showing the relationship between, e.g., palette size, execution time and SSIM.
- **Video quantization:** The system can read a video file, quantize each frame, and save the processed result as a new video file.
- **Live webcam quantization:** The system can process a webcam feed in real time. The graphical interface allows the user to select the algorithm, number of colors, and camera resolution.
- **Live palette reuse:** The `Octree-Live` mode reuses the generated palette for several frames before rebuilding it. This improves performance in live video scenarios where consecutive frames often have similar color distributions.
- **Game or window capture quantization:** The system can capture an application window, apply color quantization, and display the result as a real-time overlay. This demonstrates color quantization as a post-processing effect for interactive graphics.

4 Empirical Assessment

The empirical assessment compares the quantization algorithms from three perspectives: visual quality, computational speed, and the trade-off between these two objectives. This is important because the best-looking algorithm is not necessarily suitable for interactive or real-time use.

The experiments were evaluated over a grid of scenarios consisting of four resolutions (4K, 2K, 1920x1080, and 720p) and six target palette sizes (8, 16, 32, 64, 128, and 256 colors). For each scenario, the same algorithms were tested: Uniform, K-Means, SOM, Median-Cut, Octree-Baseline, Octree-Euclidean, Octree-K-Means, and Octree-SOM. The main text presents representative Full HD results, while the complete scenario-level results and rankings are included in the Appendix.

In addition to static image benchmarks, two live-processing experiments were performed. The first measures the average FPS when quantizing a live camera feed, where the camera itself is limited to approximately 30 FPS. The second measures the average FPS during live game or screen capture integration, where 60 FPS is used as the real-time reference.

All experiments were run on the same hardware setup to make the execution time and FPS results comparable. The benchmark machine used an AMD Ryzen 7 7840HS processor with Radeon 780M integrated graphics at 3.80 GHz, 32 GB of RAM, and an NVIDIA GeForce RTX 4060 Laptop GPU with 8 GB of VRAM.

4.1 Accuracy

The accuracy plots are generated using the Structural Similarity Index Measure (SSIM). It compares the original image x and the quantized image y using luminance, contrast, and structural similarity:

$$\mathrm{SSIM}(x, y) = \frac{(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)}.$$

The value is between 0 and 1, where higher values indicate that the quantized image is more visually similar to the original. For each image, target color count, and algorithm, the SSIM values are averaged:

$$\overline{\mathrm{SSIM}}_{a,c} = \frac{1}{n}\sum_{i=1}^{n} \mathrm{SSIM}_{a,c,i}.$$

The implementation computes SSIM between the original and processed RGB images, resizes the processed image if needed, and then averages the SSIM values by target color count and algorithm.

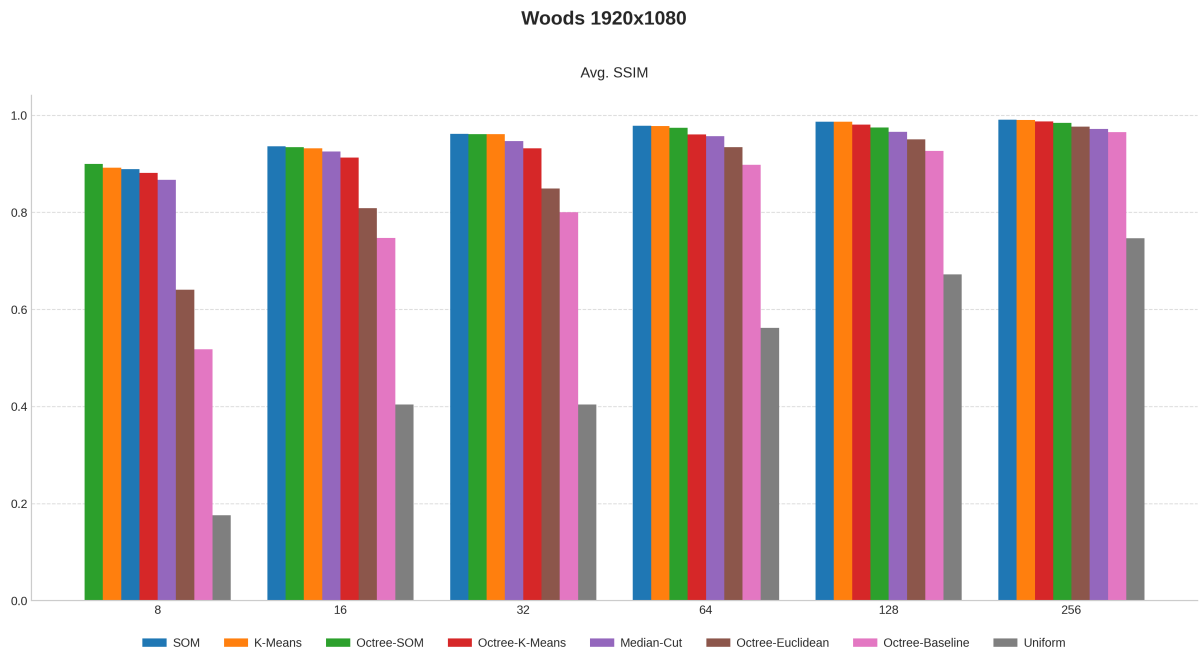

Figure 8: Average SSIM for the Woods image at 1920x1080 resolution. Higher values indicate better visual quality.

The results show that increasing the number of target colors improves the quality for all algorithms. SOM and K-Means achieve the highest visual quality, especially at larger palette sizes. However, their improved accuracy comes at a significant computational cost. Uniform quantization is clearly the weakest in visual quality, particularly when the number of colors is small.

4.2 Speed

The speed plots in Figure 4.2 are generated from the measured execution time of each algorithm. For each image, target color count, and algorithm, repeated measurements are averaged:

$$\bar{t}_{a,c} = \frac{1}{n} \sum_{i=1}^{n} t_{a,c,i},$$

where $t_{a,c,i}$ is the execution time in milliseconds for algorithm a, target color count c, and benchmark run i. Lower values indicate faster execution. The plot uses a logarithmic y-axis because the measured runtimes differ by several orders of magnitude. This makes both fast and slow algorithms visible in the same figure. The implementation groups the data by target color count and algorithm, averages `Time Taken (ms)`, and then displays the y-axis on a log scale.

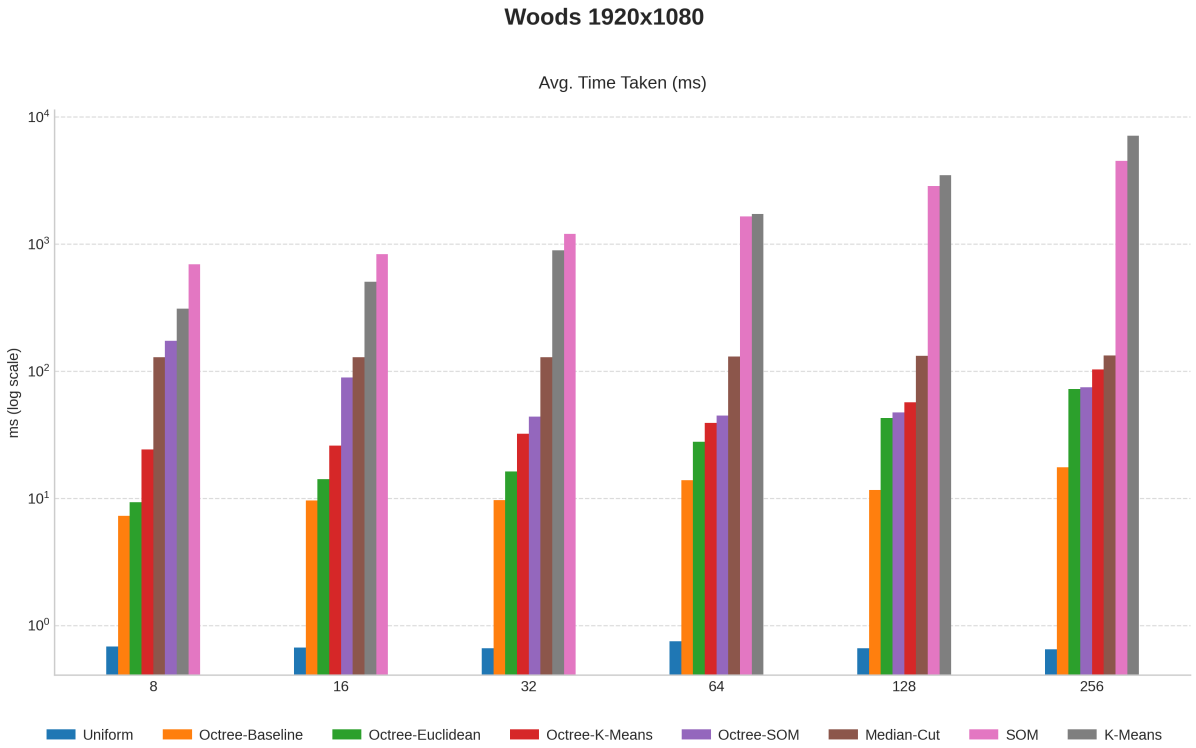

Figure 9: Average execution time for the Woods image at 1920x1080 resolution. The logarithmic scale highlights differences between both fast and slow algorithms.

Uniform quantization is the fastest method by a large margin, followed by the simpler Octree variants. K-Means and SOM are much slower because they rely on iterative optimization. This makes them less practical for real-time processing, even though they can produce high-quality results.

4.3 Trade-off

The trade-off score combines computational speed and visual quality into one lower-is-better ranking. The score is computed separately for each scenario, where a scenario is defined by a resolution and a target number of colors. This prevents results from different image sizes or palette sizes from being compared directly.

Since execution times vary by several orders of magnitude, runtime is first compressed using a logarithmic transformation:

$$t'_{a,s} = \log(1 + t_{a,s}),$$

where $t_{a,s}$ is the execution time of algorithm a in scenario s. The transformed runtime is then normalized within the scenario:

$$\tilde{t}_{a,s} = \frac{t'_{a,s} - \min_b(t'_{b,s})}{\max_b(t'_{b,s}) - \min_b(t'_{b,s})}.$$

Thus, the fastest algorithm receives a normalized runtime value of 0 and the slowest receives a value of 1.

Visual quality is normalized in the opposite direction using SSIM:

$$\tilde{q}_{a,s} = \frac{\mathrm{SSIM}_{a,s} - \min_b(\mathrm{SSIM}_{b,s})}{\max_b(\mathrm{SSIM}_{b,s}) - \min_b(\mathrm{SSIM}_{b,s})}.$$

Here, the worst-quality algorithm receives a value of 0 and the best-quality algorithm receives a value of 1. The final trade-off score is the Euclidean distance to the ideal point, where the ideal algorithm would be both the fastest and have the highest SSIM:

$$P_{a,s} = \sqrt{\tilde{t}_{a,s}^2 + (1 - \tilde{q}_{a,s})^2}.$$

A lower score therefore indicates a better balance between runtime and visual quality. The algorithms are ranked within each scenario based on this score, where rank 1 represents the best trade-off.

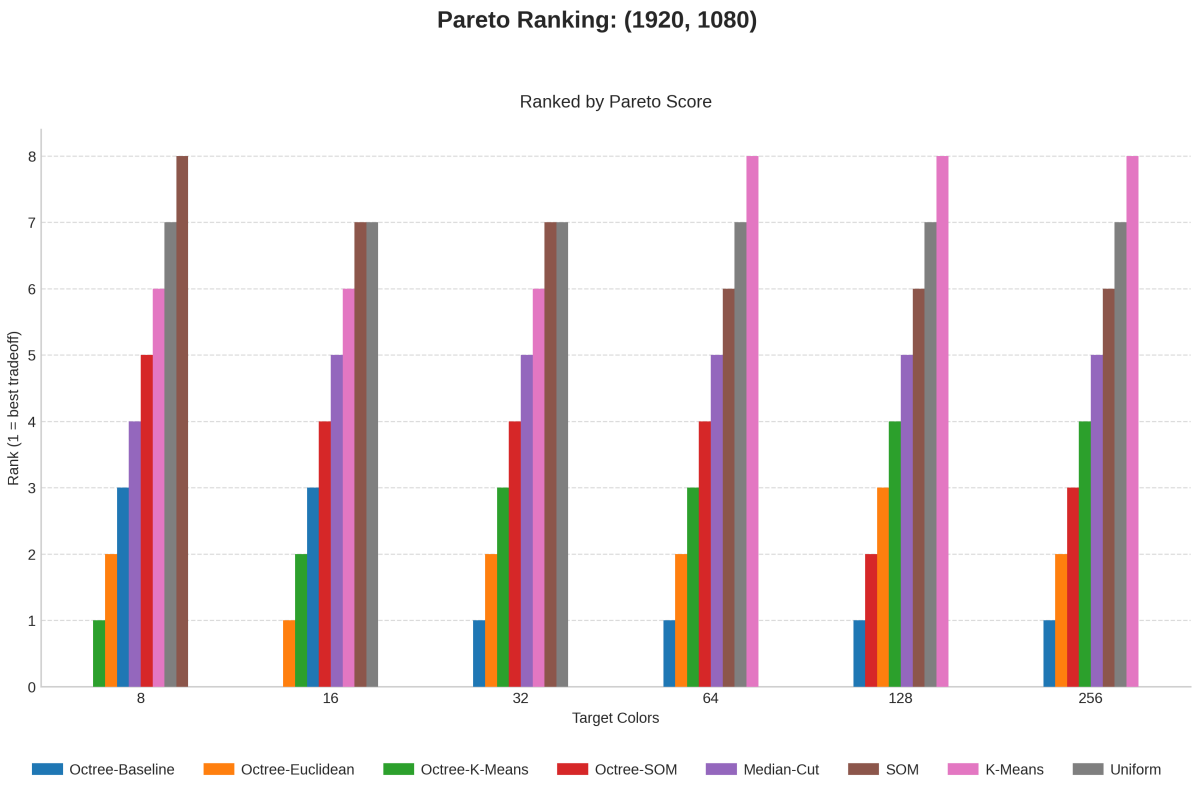

Figure 10: Trade-off ranking at 1920x1080 resolution. Rank 1 represents the algorithm closest to the ideal combination of low runtime and high SSIM.

The Octree-based methods dominate the trade-off ranking because they remain much faster than K-Means and SOM while preserving clearly better image quality than Uniform quantization. Octree-Baseline and Octree-Euclidean are especially strong because they stay close to the ideal point across several target color counts.

4.4 Real-Time Camera Feed FPS

The live camera test measures how many frames per second each algorithm can process when used on a camera stream. Since the camera feed is limited to approximately 30 FPS, values close to 30 FPS indicate real-time performance.

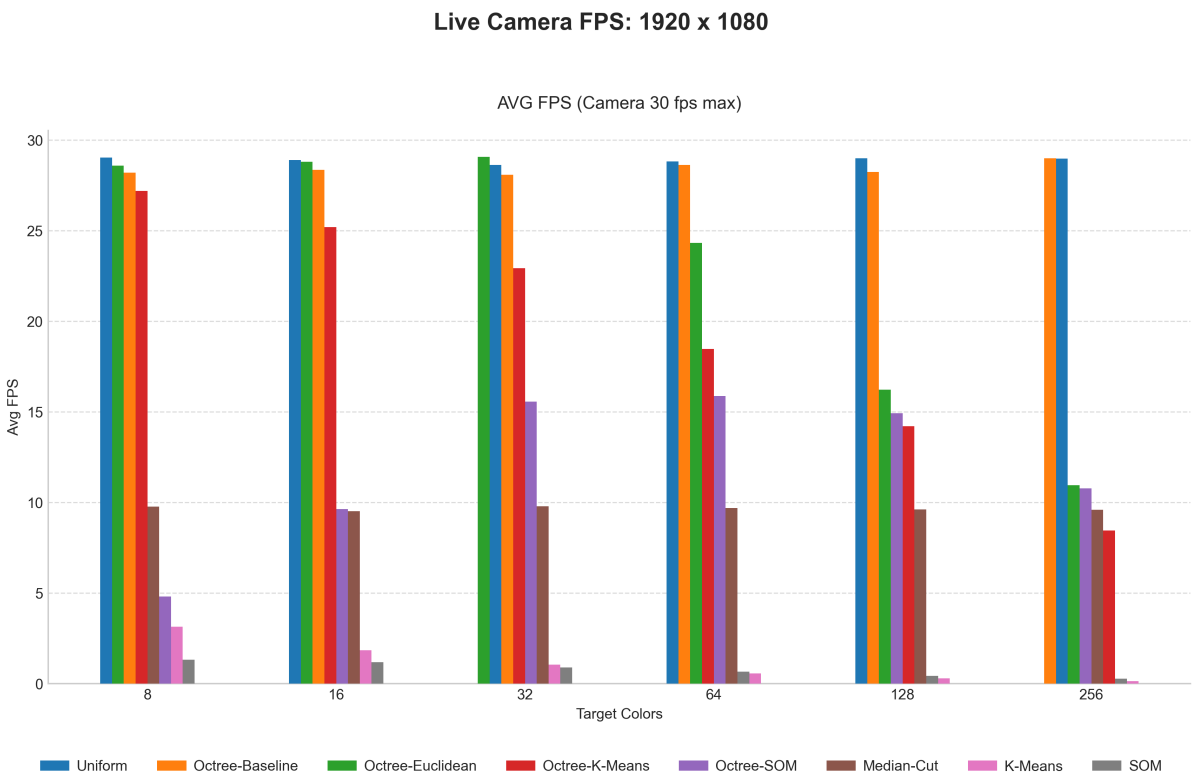

Figure 11: Average FPS for live camera quantization at 1920x1080 resolution. The camera stream is limited to approximately 30 FPS.

Uniform and Octree-Baseline are able to stay close to the camera limit across most target color counts. Octree-Euclidean also performs well at smaller palettes, but its FPS drops as the number of target colors increases. K-Means and SOM are not suitable for real-time camera use in this implementation.

4.5 Real-Time Game and Screen Capture Integration FPS

The live game and screen capture benchmark measures performance in a more demanding real-time setting. Here, 60 FPS is used as the reference target for smooth live processing.

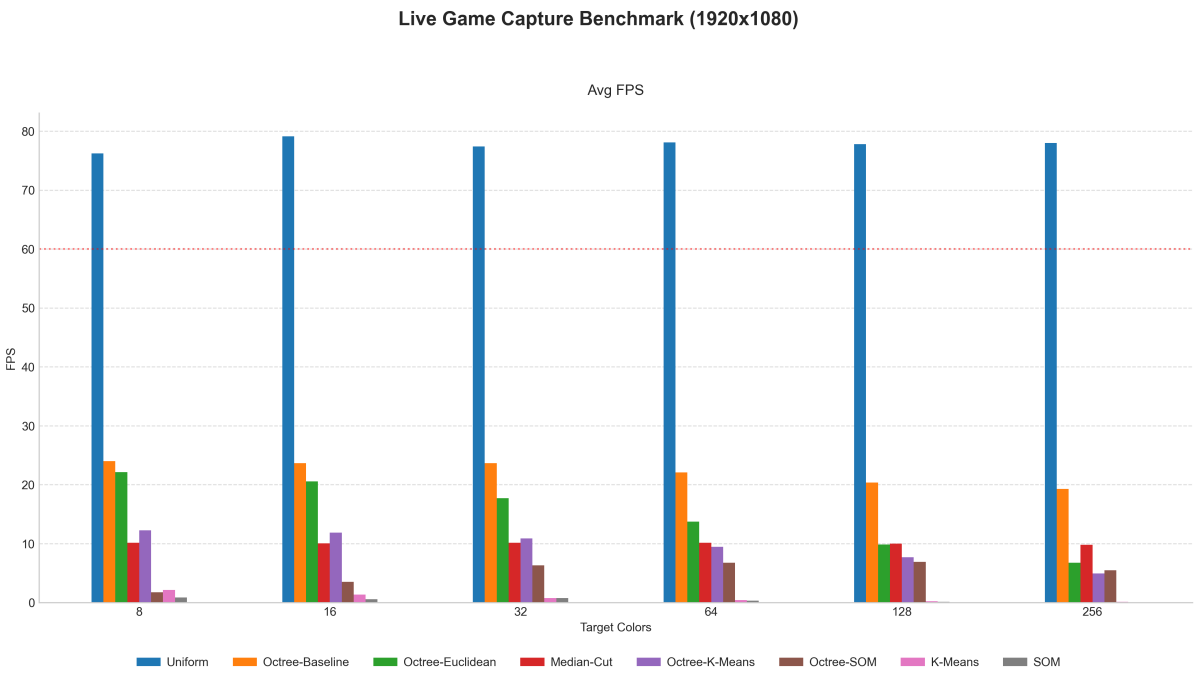

Figure 12: Average FPS for live game and screen capture quantization at 1920x1080 resolution. The red reference line marks 60 FPS.

Uniform quantization is the only method that clearly exceeds the 60 FPS target. However, this comes with the lowest visual quality. The Octree methods provide better quality but generally remain below 60 FPS in the game capture benchmark. This suggests that further optimization would be required before the higher-quality methods can be used for smooth 60 FPS live capture.

4.6 Summary of Results

Before summarizing the results, each benchmark case was treated as a separate scenario, defined by one resolution and one target color count. Within each scenario, the algorithms were ranked separately for speed, accuracy, and trade-off. Speed was ranked using execution time, where lower is better. Accuracy was ranked using SSIM, where higher is better. Trade-off was ranked using the pareto score, where lower is better.

After the scenario-level rankings were computed, the results were aggregated by algorithm across all tested resolutions and target color counts. For each category, the average rank, median rank, number of wins, worst rank, and percentage gap to the best algorithm were calculated. The final summary table reports the three algorithms with the best overall average rank in each category, using wins and percentage gaps as secondary tie-breaking criteria. The detailed summary of results is in appendix A.

Table 2: Top-performing algorithms by category. Aggregated over every resolution and target color count.

Rank	Accuracy	Speed	Trade-off
1	SOM	Uniform	Octree-Baseline
2	K-Means	Octree-Baseline	Octree-Euclidean
3	Octree-SOM	Octree-Euclidean	Octree-K-Means

Overall, SOM and K-Means produce the best image quality, but their execution times make them unsuitable for real-time use. Uniform quantization is by far the fastest, but its visual quality is weak. The Octree-based methods provide the best practical compromise: they are fast enough for interactive use while maintaining much better visual quality than Uniform quantization. Therefore, Octree-Baseline is the strongest overall choice when both quality and performance are considered.

5 Additional Practical Components

In addition to the algorithms included in the main benchmark tables, two practical components were implemented but not included in the aggregated benchmark ranking: Octree-Live and a shader-based stylization mode. These were excluded from the main comparison because they are application-specific rather than general static-image quantization algorithms.

5.1 Octree-Live

Octree-Live was implemented to test whether Octree quantization could be optimized during continuous live video processing. Instead of rebuilding the Octree palette for every frame, the palette is recomputed only after a fixed number of frames, for example every 10 frames. The frames between palette updates are then quantized using the most recently computed palette.

This reduces the amount of palette construction work during live processing and gives a small performance improvement. However, the drawback is that the palette does not always represent the exact colors of the current frame. If the scene changes quickly, the color palette may temporarily lag behind the video content. Therefore, Octree-Live is useful as a practical optimization for live camera or screen capture scenarios, but it is not directly comparable to the static-image benchmark algorithms.

5.2 Shader-Based Stylization

A shader-based mode was also included as an optional stylization approach. Unlike the other methods, the shader is not primarily designed to produce the most accurate reduced color palette. Instead, it is used to stylize the canvas in real time when the user wants a more artistic or posterized visual effect.

The shader approach is useful because it can be applied directly during rendering and can therefore provide immediate visual feedback. This makes it suitable for interactive applications where the goal is not only compression or palette reduction, but also a specific visual style. For this reason, the shader mode was treated as a separate practical feature rather than as part of the main accuracy, speed, and trade-off ranking.

6 Discussion

The results show that there is no single algorithm that is best in every category (see Table 3 in A). The highest visual quality is achieved by SOM and K-Means, but both methods are too slow for real-time use at higher resolutions. This is expected, since both rely on iterative optimization and become increasingly expensive as the number of target colors grows.

Uniform quantization is the opposite case. It is consistently the fastest method and is the only algorithm that clearly reaches the 60 FPS target in the live game capture benchmark. However, its visual quality is noticeably weaker, especially for small palette sizes. Therefore, Uniform quantization is only suitable when speed is more important than image quality.

The Octree-based algorithms provide the best practical compromise. They are much faster than SOM and K-Means, while producing significantly better visual quality than Uniform quantization. This is also reflected in the trade-off ranking, where Octree-Baseline, Octree-Euclidean, and Octree-K-Means are the strongest overall performers. In particular, Octree-Baseline is a good default choice because it performs well across different target color counts and resolutions without requiring heavy optimization.

The live FPS experiments also show that offline image quality and real-time performance should be evaluated separately. An algorithm that looks strong in static image benchmarks may still be unsuitable for live camera or screen capture use. For real-time applications, runtime stability and FPS are as important as visual quality.

One limitation of the experiments is that the results depend on the selected input images and implementation details. More images and more hardware setups would be needed for a fully general conclusion. However, the current results are sufficient to identify the main performance patterns between the algorithms.

7 Conclusion

This project compared several color quantization algorithms across different resolutions, palette sizes, and real-time processing scenarios. The algorithms were evaluated using visual quality, execution time, trade-off ranking, and live FPS performance.

The main finding is that SOM and K-Means achieve the best image quality, but their high runtime makes them impractical for real-time use. Uniform quantization is extremely fast, but loses too much visual detail to be considered a good general-purpose method. The Octree-based methods offer the best balance between these two extremes.

Overall, Octree-Baseline is the strongest general choice in this project. It achieves a good trade-off between speed and quality, performs consistently across the tested scenarios, and is much more suitable for interactive use than the iterative algorithms. For applications where maximum visual quality is required and runtime is not important, SOM or K-Means can still be preferred. For strict real-time 60 FPS processing, Uniform quantization remains the fastest option, although with reduced image quality.

In summary, the results support using Octree-based quantization as the most practical approach for high-resolution and near real-time color reduction.

References

[1] Paul S. Heckbert, 1982, Color Image Quantization for Frame Buffer Display.

[2] Oleg Verevka and John W. Buchanan, 1995, Local K-means Algorithm for Colour Image Quantization.

[3] Teuvo Kohonen, 2013, Essentials of the Self-Organizing Map.

[4] Michael Gervautz and Werner Purgathofer, 1988, A Simple Method for Color Quantization: Octree Quantization.

[5] Hyun Jun Park, Kwang Baek Kim, and Eui-Young Cha, 2016, An Effective Color Quantization Method Using Octree-Based Self-Organizing Maps.

[6] Merve Arslan and Recep Demirci, 2022, Automatic Initialization of Image Clustering Algorithms.

[7] Tom MacWright, n.d., Octree Color Quantization.

[8] Xiph.Org Foundation, n.d., Xiph.org Video Test Media [derf's collection].

[9] Rich Franzen, n.d., Kodak Lossless True Color Image Suite.

[10] University of Southern California Signal and Image Processing Institute, n.d., SIPI Image Database: Miscellaneous Volume.

A Summary of Results

Table 3: Aggregated algorithm performance by category across all scenarios. Percentages show how far each algorithm was behind the best method in the same scenario.

Category	Algorithm	Avg. Rank	Median Rank	Worst Rank	Wins	Comparisons	Avg. % Behind	Median % Behind	Worst % Behind
Speed	Uniform	1.00	1.00	1.00	24	24	0.00	0.00	0.00
Speed	Octree-Baseline	2.00	2.00	2.00	0	24	1706.45	1669.48	2900.31
Speed	Octree-Euclidean	3.04	3.00	4.00	0	24	4556.86	3253.17	11529.94
Speed	Octree-K-Means	4.29	4.00	6.00	0	24	6779.28	5059.88	20178.71
Speed	Octree-SOM	5.00	5.00	7.00	0	24	16664.77	11381.65	84576.80
Speed	Median-Cut	5.71	6.00	6.00	0	24	18569.80	18315.98	20843.49
Speed	K-Means	7.46	7.50	8.00	0	24	350523.70	186758.45	1128997.40
Speed	SOM	7.50	7.50	8.00	0	24	275307.84	200742.41	717922.86
Accuracy	SOM	1.54	1.00	3.00	15	24	0.23	0.00	1.59
Accuracy	K-Means	1.92	2.00	3.00	5	24	0.14	0.07	0.87
Accuracy	Octree-SOM	2.88	3.00	4.00	4	24	0.58	0.63	1.28
Accuracy	Octree-K-Means	4.00	4.00	5.00	0	24	1.88	1.85	4.34
Accuracy	Median-Cut	4.83	5.00	6.00	0	24	2.29	2.19	4.47
Accuracy	Octree-Euclidean	5.83	6.00	6.00	0	24	11.21	8.42	30.56
Accuracy	Octree-Baseline	7.00	7.00	7.00	0	24	16.75	12.86	44.78
Accuracy	Uniform	8.00	8.00	8.00	0	24	49.94	50.65	83.20
Trade-off	Octree-Baseline	1.79	1.00	3.00	14	24	7.19	0.00	32.51
Trade-off	Octree-Euclidean	1.96	2.00	3.00	5	24	17.18	11.80	51.36
Trade-off	Octree-K-Means	2.63	3.00	5.00	5	24	21.94	14.39	74.57
Trade-off	Octree-SOM	3.96	4.00	6.00	0	24	47.45	45.93	103.40
Trade-off	Median-Cut	4.71	5.00	5.00	0	24	63.16	62.97	77.19
Trade-off	SOM	6.88	6.50	8.00	0	24	154.11	146.57	226.37
Trade-off	K-Means	6.96	7.00	8.00	0	24	151.48	142.47	246.92
Trade-off	Uniform	7.00	7.00	7.00	0	24	159.78	146.57	246.92

B Detailed Results

Table 4: Detailed algorithm results per scenario.

Res.	C	Algorithm	ms	SSIM	Tr.	S%	A%	T%	SR	AR	TR
1280x720	8	Octree-K-Means	13.8	0.867	0.460	3970%	3%	0%	4	4	1
1280x720	8	Octree-Euclidean	5.7	0.617	0.479	1573%	31%	4%	3	6	2
1280x720	8	Octree-Baseline	5.0	0.491	0.610	1378%	45%	33%	2	7	3
1280x720	8	Median-Cut	54.4	0.852	0.713	15935%	4%	55%	5	5	4
1280x720	8	Octree-SOM	82.5	0.881	0.790	24228%	1%	72%	6	3	5
1280x720	8	K-Means	133.8	0.889	0.882	39372%	0%	92%	7	1	6
1280x720	8	Uniform	0.3	0.149	1.000	0%	83%	117%	1	8	7
1280x720	8	SOM	249.3	0.881	1.000	73414%	1%	117%	8	2	8
1280x720	16	Octree-Euclidean	7.4	0.798	0.419	2265%	15%	0%	3	6	1
1280x720	16	Octree-K-Means	15.2	0.895	0.471	4761%	4%	12%	4	5	2
1280x720	16	Octree-Baseline	7.1	0.735	0.487	2167%	21%	16%	2	7	3
1280x720	16	Octree-SOM	42.0	0.929	0.647	13327%	1%	54%	5	3	4
1280x720	16	Median-Cut	54.1	0.922	0.693	17191%	1%	65%	6	4	5
1280x720	16	K-Means	219.7	0.932	0.949	70070%	0%	126%	7	2	6
1280x720	16	SOM	289.2	0.935	1.000	92251%	0%	138%	8	1	7
1280x720	16	Uniform	0.3	0.365	1.000	0%	61%	138%	1	8	7
1280x720	32	Octree-Euclidean	9.4	0.842	0.406	2879%	12%	0%	3	6	1
1280x720	32	Octree-Baseline	6.4	0.792	0.410	1927%	18%	1%	2	7	2
1280x720	32	Octree-K-Means	17.8	0.935	0.459	5533%	3%	13%	4	5	3
1280x720	32	Octree-SOM	21.6	0.960	0.489	6749%	0%	20%	5	1	4
1280x720	32	Median-Cut	52.5	0.944	0.637	16548%	2%	57%	6	4	5
1280x720	32	K-Means	396.2	0.959	0.981	125613%	0%	142%	7	3	6
1280x720	32	Uniform	0.3	0.365	1.000	0%	62%	146%	1	8	7
1280x720	32	SOM	441.7	0.960	1.000	140050%	0%	146%	8	2	8
1280x720	64	Octree-Baseline	6.4	0.891	0.331	1896%	9%	0%	2	7	1
1280x720	64	Octree-Euclidean	13.6	0.928	0.392	4164%	5%	18%	3	6	2
1280x720	64	Octree-SOM	20.5	0.971	0.438	6339%	1%	32%	4	3	3
1280x720	64	Octree-K-Means	22.3	0.961	0.452	6897%	2%	36%	5	4	4
1280x720	64	Median-Cut	54.0	0.955	0.587	16832%	2%	77%	6	5	5
1280x720	64	SOM	757.6	0.977	0.997	237491%	0%	201%	7	1	6
1280x720	64	Uniform	0.3	0.529	1.000	0%	46%	202%	1	8	7
1280x720	64	K-Means	773.0	0.976	1.000	242341%	0%	202%	8	2	8
1280x720	128	Octree-Baseline	7.1	0.923	0.316	2063%	6%	0%	2	7	1
1280x720	128	Octree-Euclidean	19.1	0.948	0.401	5747%	4%	27%	3	6	2
1280x720	128	Octree-SOM	23.1	0.974	0.413	6988%	1%	30%	4	4	3
1280x720	128	Octree-K-Means	30.0	0.978	0.448	9108%	1%	42%	5	3	4
1280x720	128	Median-Cut	55.5	0.963	0.536	16892%	2%	69%	6	5	5
1280x720	128	SOM	1219.6	0.986	0.967	373618%	0%	206%	7	1	6
1280x720	128	Uniform	0.3	0.648	1.000	0%	34%	216%	1	8	7
1280x720	128	K-Means	1537.5	0.985	1.000	471033%	0%	216%	8	2	8
1280x720	256	Octree-Baseline	9.6	0.962	0.288	2900%	3%	0%	2	7	1
1280x720	256	Octree-SOM	36.7	0.982	0.432	11408%	1%	50%	3	4	2
1280x720	256	Octree-Euclidean	37.0	0.975	0.436	11530%	2%	51%	4	5	3
1280x720	256	Median-Cut	56.7	0.970	0.492	17694%	2%	71%	5	6	4
1280x720	256	Octree-K-Means	64.6	0.984	0.503	20179%	1%	75%	6	3	5
1280x720	256	SOM	1976.2	0.990	0.941	620354%	0%	226%	7	1	6
1280x720	256	Uniform	0.3	0.729	1.000	0%	26%	247%	1	8	7
1280x720	256	K-Means	3132.4	0.990	1.000	983346%	0%	247%	8	2	8
1920x1080	8	Octree-K-Means	24.2	0.881	0.450	3459%	2%	0%	4	4	1
1920x1080	8	Octree-Euclidean	9.3	0.640	0.468	1269%	29%	4%	3	6	2
1920x1080	8	Octree-Baseline	7.3	0.518	0.590	966%	42%	31%	2	7	3
1920x1080	8	Median-Cut	128.7	0.867	0.723	18821%	4%	61%	5	5	4
1920x1080	8	Octree-SOM	173.0	0.900	0.770	25344%	0%	71%	6	1	5
1920x1080	8	K-Means	310.9	0.892	0.867	45621%	1%	93%	7	2	6
1920x1080	8	Uniform	0.7	0.176	1.000	0%	80%	122%	1	8	7
1920x1080	8	SOM	692.4	0.889	1.000	101719%	1%	122%	8	3	8
1920x1080	16	Octree-Euclidean	14.1	0.809	0.428	2008%	14%	0%	3	6	1
1920x1080	16	Octree-K-Means	25.9	0.913	0.450	3763%	2%	5%	4	5	2

Continued on next page

Table 4: Detailed algorithm selection results per scenario (continued).

Res.	C	Algorithm	ms	SSIM	Tr.	S%	A%	T%	SR	AR	TR
1920x1080	16	Octree-Baseline	9.6	0.747	0.464	1336%	20%	8%	2	7	3
1920x1080	16	Octree-SOM	89.1	0.934	0.642	13178%	0%	50%	5	2	4
1920x1080	16	Median-Cut	128.5	0.925	0.701	19065%	1%	64%	6	4	5
1920x1080	16	K-Means	504.5	0.932	0.920	75110%	0%	115%	7	3	6
1920x1080	16	SOM	831.4	0.936	1.000	123847%	0%	134%	8	1	7
1920x1080	16	Uniform	0.7	0.404	1.000	0%	57%	134%	1	8	7
1920x1080	32	Octree-Baseline	9.7	0.800	0.405	1369%	17%	0%	2	7	1
1920x1080	32	Octree-Euclidean	16.3	0.849	0.409	2367%	12%	1%	3	6	2
1920x1080	32	Octree-K-Means	32.1	0.932	0.458	4777%	3%	13%	4	5	3
1920x1080	32	Octree-SOM	43.8	0.961	0.501	6547%	0%	24%	5	2	4
1920x1080	32	Median-Cut	128.4	0.947	0.662	19376%	2%	64%	6	4	5
1920x1080	32	K-Means	891.6	0.961	0.955	135171%	0%	136%	7	3	6
1920x1080	32	SOM	1198.0	0.962	1.000	181655%	0%	147%	8	1	7
1920x1080	32	Uniform	0.7	0.404	1.000	0%	58%	147%	1	8	7
1920x1080	64	Octree-Baseline	13.8	0.898	0.366	1749%	8%	0%	2	7	1
1920x1080	64	Octree-Euclidean	27.9	0.934	0.420	3627%	4%	15%	3	6	2
1920x1080	64	Octree-K-Means	39.2	0.960	0.457	5136%	2%	25%	4	4	3
1920x1080	64	Octree-SOM	44.6	0.974	0.473	5865%	0%	29%	5	3	4
1920x1080	64	Median-Cut	129.9	0.957	0.628	17251%	2%	72%	6	5	5
1920x1080	64	SOM	1643.3	0.978	0.993	219447%	0%	172%	7	1	6
1920x1080	64	Uniform	0.7	0.562	1.000	0%	43%	174%	1	8	7
1920x1080	64	K-Means	1721.1	0.978	1.000	229846%	0%	174%	8	2	8
1920x1080	128	Octree-Baseline	11.6	0.926	0.328	1657%	6%	0%	2	7	1
1920x1080	128	Octree-SOM	47.4	0.975	0.443	7075%	1%	35%	4	4	2
1920x1080	128	Octree-Euclidean	42.8	0.951	0.443	6376%	4%	35%	3	6	3
1920x1080	128	Octree-K-Means	57.0	0.981	0.465	8517%	1%	42%	5	3	4
1920x1080	128	Median-Cut	132.1	0.965	0.577	19869%	2%	76%	6	5	5
1920x1080	128	SOM	2851.2	0.987	0.974	431035%	0%	197%	7	1	6
1920x1080	128	Uniform	0.7	0.672	1.000	0%	32%	205%	1	8	7
1920x1080	128	K-Means	3474.9	0.986	1.000	525352%	0%	205%	8	2	8
1920x1080	256	Octree-Baseline	17.5	0.965	0.308	2602%	3%	0%	2	7	1
1920x1080	256	Octree-Euclidean	72.2	0.977	0.457	11054%	1%	49%	3	5	2
1920x1080	256	Octree-SOM	74.4	0.984	0.458	11404%	1%	49%	4	4	3
1920x1080	256	Octree-K-Means	103.0	0.987	0.496	15818%	0%	61%	5	3	4
1920x1080	256	Median-Cut	133.2	0.971	0.532	20481%	2%	73%	6	6	5
1920x1080	256	SOM	4508.7	0.991	0.946	696750%	0%	207%	7	1	6
1920x1080	256	Uniform	0.6	0.747	1.000	0%	25%	225%	1	8	7
1920x1080	256	K-Means	7092.7	0.990	1.000	1096133%	0%	225%	8	2	8
2560x1440	8	Octree-K-Means	37.8	0.875	0.460	3212%	2%	0%	4	4	1
2560x1440	8	Octree-Euclidean	16.3	0.632	0.489	1332%	29%	6%	3	6	2
2560x1440	8	Octree-Baseline	12.6	0.515	0.598	1004%	42%	30%	2	7	3
2560x1440	8	Median-Cut	234.1	0.860	0.747	20433%	4%	62%	5	5	4
2560x1440	8	Octree-SOM	401.2	0.891	0.831	35086%	0%	81%	6	1	5
2560x1440	8	K-Means	532.3	0.888	0.876	46581%	0%	90%	7	2	6
2560x1440	8	Uniform	1.1	0.171	1.000	0%	81%	117%	1	8	7
2560x1440	8	SOM	1167.8	0.882	1.000	102312%	1%	117%	8	3	8
2560x1440	16	Octree-Euclidean	26.7	0.799	0.459	2217%	14%	0%	3	6	1
2560x1440	16	Octree-K-Means	43.0	0.906	0.462	3630%	3%	1%	4	5	2
2560x1440	16	Octree-Baseline	16.9	0.738	0.481	1366%	21%	5%	2	7	3
2560x1440	16	Octree-SOM	211.0	0.922	0.700	18204%	1%	53%	5	3	4
2560x1440	16	Median-Cut	234.2	0.920	0.716	20213%	1%	56%	6	4	5
2560x1440	16	K-Means	876.2	0.931	0.917	75900%	0%	100%	7	1	6
2560x1440	16	Uniform	1.2	0.389	1.000	0%	58%	118%	1	8	7
2560x1440	16	SOM	1514.0	0.930	1.000	131220%	0%	118%	8	2	8
2560x1440	32	Octree-Baseline	15.6	0.791	0.418	1254%	18%	0%	2	7	1
2560x1440	32	Octree-Euclidean	31.2	0.840	0.444	2602%	13%	6%	3	6	2
2560x1440	32	Octree-K-Means	47.5	0.927	0.454	4015%	3%	9%	4	5	3
2560x1440	32	Octree-SOM	96.0	0.958	0.551	8216%	0%	32%	5	3	4
2560x1440	32	Median-Cut	233.0	0.942	0.679	20085%	2%	62%	6	4	5
2560x1440	32	K-Means	1588.2	0.960	0.955	137485%	0%	128%	7	1	6
2560x1440	32	Uniform	1.2	0.389	1.000	0%	60%	139%	1	8	7
2560x1440	32	SOM	2165.4	0.958	1.000	187485%	0%	139%	8	2	8
2560x1440	64	Octree-Baseline	21.7	0.889	0.383	1854%	9%	0%	2	7	1
2560x1440	64	Octree-Euclidean	50.0	0.927	0.452	4397%	5%	18%	3	6	2
2560x1440	64	Octree-K-Means	61.4	0.958	0.466	5420%	2%	22%	4	4	3
2560x1440	64	Octree-SOM	85.4	0.972	0.509	7577%	0%	33%	5	3	4
2560x1440	64	Median-Cut	233.0	0.954	0.648	20843%	2%	69%	6	5	5
2560x1440	64	SOM	2382.0	0.977	0.964	214000%	0%	152%	7	1	6
2560x1440	64	Uniform	1.1	0.545	1.000	0%	44%	161%	1	8	7
2560x1440	64	K-Means	3092.9	0.975	1.000	277900%	0%	161%	8	2	8
2560x1440	128	Octree-Baseline	20.0	0.913	0.361	1682%	7%	0%	2	7	1
2560x1440	128	Octree-K-Means	82.0	0.979	0.460	7211%	1%	27%	4	3	2
2560x1440	128	Octree-Euclidean	71.5	0.940	0.463	6268%	5%	28%	3	6	3
2560x1440	128	Octree-SOM	86.2	0.973	0.467	7585%	1%	29%	5	4	4
2560x1440	128	Median-Cut	234.0	0.962	0.594	20753%	2%	65%	6	5	5
2560x1440	128	SOM	4347.1	0.985	0.955	387280%	0%	165%	7	1	6
2560x1440	128	Uniform	1.1	0.655	1.000	0%	34%	177%	1	8	7
2560x1440	128	K-Means	6225.0	0.985	1.000	554632%	0%	177%	8	2	8
2560x1440	256	Octree-Baseline	27.8	0.961	0.319	2286%	3%	0%	2	7	1
2560x1440	256	Octree-Euclidean	128.5	0.973	0.476	10939%	2%	50%	3	5	2
2560x1440	256	Octree-SOM	133.4	0.982	0.477	11359%	1%	50%	4	4	3
2560x1440	256	Octree-K-Means	156.3	0.986	0.495	13327%	0%	55%	5	3	4
2560x1440	256	Median-Cut	238.3	0.968	0.549	20371%	2%	72%	6	6	5
2560x1440	256	SOM	7258.3	0.990	0.937	623348%	0%	194%	7	1	6
2560x1440	256	Uniform	1.2	0.731	1.000	0%	26%	214%	1	8	7
2560x1440	256	K-Means	12575.9	0.989	1.000	1080098%	0%	214%	8	2	8
3840x2160	8	Octree-K-Means	81.4	0.874	0.472	3090%	3%	0%	4	4	1
3840x2160	8	Octree-Euclidean	36.1	0.636	0.513	1313%	29%	9%	3	6	2
3840x2160	8	Octree-Baseline	26.0	0.526	0.612	918%	41%	30%	2	7	3
3840x2160	8	Median-Cut	459.3	0.857	0.731	17897%	4%	55%	5	5	4
3840x2160	8	K-Means	1296.8	0.894	0.884	50710%	0%	87%	6	2	5
3840x2160	8	Octree-SOM	2161.2	0.897	0.961	84577%	0%	103%	7	1	6
3840x2160	8	Uniform	2.6	0.199	1.000	0%	78%	112%	1	8	7
3840x2160	8	SOM	2805.4	0.883	1.000	109815%	2%	112%	8	3	8
3840x2160	16	Octree-Euclidean	53.0	0.799	0.478	1942%	14%	0%	3	6	1
3840x2160	16	Octree-K-Means	90.2	0.901	0.482	3375%	4%	1%	4	5	2
3840x2160	16	Octree-Baseline	37.9	0.734	0.522	1359%	21%	9%	2	7	3
3840x2160	16	Median-Cut	456.8	0.921	0.717	17510%	1%	50%	5	4	4
3840x2160	16	Octree-SOM	998.7	0.932	0.832	38397%	0%	74%	6	2	5
3840x2160	16	K-Means	2021.0	0.934	0.936	77802%	0%	96%	7	1	6
3840x2160	16	Uniform	2.6	0.416	1.000	0%	55%	109%	1	8	7
3840x2160	16	SOM	3120.9	0.931	1.000	120198%	0%	109%	8	3	8
3840x2160	32	Octree-K-Means	100.2	0.930	0.479	3874%	3%	0%	4	5	1

Continued on next page

Table 4: Detailed algorithm selection results per scenario (continued).

Res.	C	Algorithm	ms	SSIM	Tr.	S%	A%	T%	SR	AR	TR
3840x2160	32	Octree-Euclidean	64.2	0.806	0.501	2444%	16%	5%	3	6	2
3840x2160	32	Octree-Baseline	37.4	0.741	0.527	1383%	23%	10%	2	7	3
3840x2160	32	Median-Cut	459.5	0.939	0.691	18122%	2%	44%	5	4	4
3840x2160	32	Octree-SOM	466.6	0.952	0.693	18402%	1%	45%	6	3	5
3840x2160	32	K-Means	3625.6	0.960	0.983	143671%	0%	105%	7	1	6
3840x2160	32	Uniform	2.5	0.416	1.000	0%	57%	109%	1	8	7
3840x2160	32	SOM	4092.5	0.958	1.000	162190%	0%	109%	8	2	8
3840x2160	64	Octree-Baseline	50.3	0.889	0.411	1886%	9%	0%	2	7	1
3840x2160	64	Octree-Euclidean	113.0	0.926	0.473	4365%	5%	15%	3	6	2
3840x2160	64	Octree-K-Means	128.6	0.967	0.475	4984%	1%	16%	4	4	3
3840x2160	64	Octree-SOM	273.9	0.970	0.574	10724%	1%	40%	5	3	4
3840x2160	64	Median-Cut	456.8	0.953	0.643	17954%	2%	57%	6	5	5
3840x2160	64	SOM	6314.5	0.976	0.986	249448%	0%	140%	7	1	6
3840x2160	64	Uniform	2.5	0.567	1.000	0%	42%	143%	1	8	7
3840x2160	64	K-Means	7001.0	0.976	1.000	276580%	0%	143%	8	2	8
3840x2160	128	Octree-Baseline	43.5	0.918	0.369	1524%	7%	0%	2	7	1
3840x2160	128	Octree-K-Means	175.4	0.979	0.470	6448%	1%	27%	4	3	2
3840x2160	128	Octree-Euclidean	155.7	0.944	0.473	5711%	4%	28%	3	6	3
3840x2160	128	Octree-SOM	227.1	0.975	0.502	8379%	1%	36%	5	4	4
3840x2160	128	Median-Cut	458.9	0.961	0.590	17029%	2%	60%	6	5	5
3840x2160	128	SOM	8321.6	0.985	0.937	310538%	0%	154%	7	1	6
3840x2160	128	Uniform	2.7	0.670	1.000	0%	32%	171%	1	8	7
3840x2160	128	K-Means	14018.7	0.985	1.000	523206%	0%	171%	8	2	8
3840x2160	256	Octree-Baseline	63.5	0.961	0.343	2428%	3%	0%	2	7	1
3840x2160	256	Octree-Euclidean	278.4	0.972	0.491	10976%	2%	43%	3	5	2
3840x2160	256	Octree-K-Means	309.2	0.986	0.498	12200%	0%	45%	4	3	3
3840x2160	256	Octree-SOM	329.2	0.982	0.506	12997%	1%	47%	5	4	4
3840x2160	256	Median-Cut	467.8	0.967	0.551	18510%	2%	60%	6	6	5
3840x2160	256	SOM	18048.8	0.989	0.950	717923%	0%	177%	7	1	6
3840x2160	256	Uniform	2.5	0.741	1.000	0%	25%	191%	1	8	7
3840x2160	256	K-Means	28381.9	0.989	1.000	1128997%	0%	191%	8	2	8